



# Probability and Statistics for Computer Science

## Programming Laboratory Manual

Rekha R, Assistant Professor

Department of Computer Engineering, FoT, University of Delhi

### Contents

|          |                                                                              |           |
|----------|------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Understanding Jupyter Notebook as an Interactive Interpreter</b>          | <b>3</b>  |
| 1.1      | Jupyter Notebook in Windows . . . . .                                        | 3         |
| 1.2      | Jupyter Notebook in a Virtual Environment (Ubuntu) . . . . .                 | 4         |
| 1.3      | Using Jupyter Notebook . . . . .                                             | 6         |
| 1.3.1    | Notebook Structure . . . . .                                                 | 6         |
| 1.3.2    | Working with Code Cells . . . . .                                            | 6         |
| 1.3.3    | Running Code . . . . .                                                       | 6         |
| <b>2</b> | <b>Python Basics</b>                                                         | <b>8</b>  |
| 2.1      | Compute the Mean (Average) . . . . .                                         | 8         |
| 2.2      | Compute the Variance and Standard Deviation . . . . .                        | 8         |
| 2.3      | Estimate Empirical Probability . . . . .                                     | 8         |
| 2.4      | Grade Classification and Probability Estimation . . . . .                    | 8         |
| 2.5      | Count Failures or Absentees . . . . .                                        | 9         |
| 2.6      | Identify Invalid or High Values . . . . .                                    | 9         |
| 2.7      | Rainfall Categorization Using Combined Conditions . . . . .                  | 9         |
| 2.8      | Estimate Conditional Probability . . . . .                                   | 10        |
| 2.9      | Empirical Distribution Function (PMF) . . . . .                              | 10        |
| 2.10     | Empirical Cumulative Distribution Function (CDF) Using Lists . . . . .       | 10        |
| 2.11     | Function to Run a Monte Carlo Experiment . . . . .                           | 10        |
| <b>3</b> | <b>Randomized Test Generation from Question Bank</b>                         | <b>12</b> |
| 3.1      | Solution . . . . .                                                           | 13        |
| <b>4</b> | <b>Bike Rental Forecasting with Discrete and Continuous Random Variables</b> | <b>18</b> |
| 4.1      | Solution . . . . .                                                           | 20        |
| <b>5</b> | <b>Website Session Analysis using Binomial and Normal Distributions</b>      | <b>21</b> |
| 5.1      | Solution . . . . .                                                           | 22        |

|          |                                                                                         |           |
|----------|-----------------------------------------------------------------------------------------|-----------|
| <b>6</b> | <b>Analysis of Medical Data through the Lens of Bivariate Normal Distribution</b>       | <b>25</b> |
| 6.1      | Solution . . . . .                                                                      | 26        |
| <b>7</b> | <b>Understanding Capital-Gain Distributions Through the Central Limit Theorem (CLT)</b> | <b>29</b> |

© 2025 Rekha R. CE, FoT, UoD

## 1. Understanding Jupyter Notebook as an Interactive Interpreter

Jupyter Notebook is an interactive web-based environment for writing Python code, visualizing data, and documenting analysis with text, equations, and plots. Unlike a traditional Python script that runs top to bottom in a file, Jupyter executes code in *cells*, which act like mini-interpreters.

### Key features:

- You can run code in parts (cell by cell).
- Output appears directly below each cell.
- Variables and functions persist across cells.
- You can combine code, text, and equations in one document.

### 1.1. Jupyter Notebook in Windows

(Contributed by: *Vinoth N., Lab Assistant*)

#### Step 1: Install Python

1. Download Python 3.11 (Windows installer, 64-bit) from: <https://www.python.org/downloads/windows/>
2. During installation:
  - Check "Add python.exe to PATH".
  - Click Install Now.

#### Step 2: Verify Python Installation

1. Open **Command Prompt** (Press Windows + R, type cmd, press Enter).
2. Type:

```
python --version  
pip --version
```

3. If both commands work, Python is installed correctly.

#### Step 3: Install Jupyter Notebook

```
pip install jupyter
```

#### Step 4: Create a Working Folder

1. Create a folder where you want to save your notebooks.
2. Right-click inside the folder and select "Open in Terminal" or "Open Command Prompt here".

## Step 5: Launch Jupyter Notebook

```
jupyter notebook
```

This will open Jupyter in your default web browser.

## Step 6: Start Creating Notebooks

- Click New → Python 3 (ipykernel) to create your first notebook.

**Note:** If jupyter is not found, close and reopen Command Prompt after installation.

## 1.2. Jupyter Notebook in a Virtual Environment (Ubuntu)

A virtual environment isolates your Python setup from system-wide installations. This avoids version conflicts and broken dependencies while giving you a clean, controlled space for your lab experiments.

### Installation

1. Install required tools. Open your terminal and run:

```
sudo apt update
sudo apt install python3-venv python3-pip -y
```

2. Create a virtual environment Navigate to your working folder (e.g., Documents/lab) and run:

```
python3 -m venv jupyterenv
```

This will create a folder named `jupyterenv/`

3. Activate the environment

```
source jupyterenv/bin/activate
```

Your terminal prompt will change something like the following, indicating that the environment is active.

```
(jupyterenv) name@yourmachine:~$
```

4. Install Jupyter Notebook inside the virtual environment

```
pip install --upgrade pip
pip install notebook
```

5. Install the Jupyter kernel package and common lab libraries

```
pip install ipykernel
pip install numpy pandas matplotlib scipy
```

6. Register the virtual environment with Jupyter.

```
python -m ipykernel install --user --name=jupyterenv --display-name
"Python (jupyterenv)"
```

This makes the environment available as a selectable kernel in Jupyter.

7. Start Jupyter Notebook

```
jupyter notebook
```

It will open in your browser at <http://localhost:8888>.

8. Select the correct kernel. In the notebook interface

- Go to **Kernel** → **Change Kernel**
- Select: **Python (jupyterenv)**

9. Verify the correct kernel. Inside a code cell, type

```
!which python
```

You should see something like the following which confirms you are using the virtual environment.

```
/home/username/jupyterenv/bin/python
```

10. Run code with installed libraries. Once the correct kernel is selected, try:

```
import numpy as np
data = np.random.normal(50, 10, size=5)
print("Sample:", data)
```

You should see a random sample output without any import errors.

11. Exiting and reusing the environment.

- To exit the notebook: click **Files**→**Shutdown**.
- To deactivate the environment:

```
deactivate
```

- To use it again later:

```
cd path/to/jupyterenv/
source jupyterenv/bin/activate
jupyter notebook
```

**Note:** Keep your `jupyterenv` folder backed up along with your lab notebooks.

### 1.3. Using Jupyter Notebook

#### 1.3.1 Notebook Structure

- **Code Cells:** Used to write and run Python code.
- **Markdown Cells:** Used to write explanations, headings, and mathematical formulas using LaTeX.

#### Example: Python Code Cell

```
import numpy as np
data = np.random.normal(50, 10, size=100)
print("Sample mean:", np.mean(data))
```

#### Example: Markdown Cell

### Example: Python Code Cell for Calculating the Sample Mean

This code imports the NumPy library and generates 100 random numbers from a normal distribution with a mean of 50 and standard deviation of 10.

It then computes and prints the **sample mean** of the generated data.

This helps demonstrate how continuous random variables can be simulated and analyzed using Python.

In your notebook, place this Markdown cell right above your code cell.

#### 1.3.2 Working with Code Cells

Each cell behaves like a mini Python console.

*Example:*

```
# Cell 1
a = 5
b = 10

# Cell 2
print("Sum is:", a + b)
```

After running Cell 1 (with **Shift + Enter**), variables **a** and **b** are remembered by the kernel. Cell 2 can use them without redefining.

#### 1.3.3 Running Code

- **Shift + Enter** — Run cell and move to the next
- **Ctrl + Enter** — Run cell and stay in place
- Use the **Run** button in the toolbar

## Recommended Libraries

- `numpy` – for numerical operations and random sampling.
- `pandas` – for data loading and manipulation.
- `matplotlib` – for visualizations.
- `scipy.stats` – for statistical distributions and tests.

## Best Practices

- Use Markdown cells to explain each step.
- Keep code modular and commented.
- Save your notebook frequently.
- Use clear variable names and descriptive plots.
- Always run cells in correct order from top to bottom when restarting.

© 2025 Rekha R. CE, FoT, UoD

## 2. Python Basics

This section introduces fundamental Python programming constructs such as `print()`, `input()`, variables, data types (`int`, `float`, `str`, `bool`), arithmetic operators (`+`, `-`, `*`, `/`, `**`), and type conversion functions (`int()`, `float()`, `str()`). It also covers lists and their operations, user-defined functions, and basic data visualization. These concepts are presented through practical problems drawn from introductory probability and statistics.

### 2.1. Compute the Mean (Average)

Given a list of numbers: [12, 15, 20, 22, 24], write a Python program to compute the mean using the formula:

$$\text{Mean} = \frac{1}{n} \sum_{i=1}^n x_i$$

*Do not use `sum()` or `mean()`. Use a `for` loop to compute the sum manually.*

### 2.2. Compute the Variance and Standard Deviation

Using the same data from Q2.1, write a Python program to compute the variance and standard deviation using the formulas:

$$\text{Variance} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad \text{Standard Deviation} = \sqrt{\text{Variance}}$$

First calculate the mean manually, then use it to compute the variance and standard deviation.

### 2.3. Estimate Empirical Probability

Given a list of binary outcomes (e.g., success/failure): [1, 0, 1, 1, 0, 0, 1, 1], write a program to estimate the probability of success (1) using:

$$P(\text{success}) = \frac{\text{Number of successes}}{\text{Total trials}}$$

### 2.4. Grade Classification and Probability Estimation

You are given a list of student scores out of 100: [85, 72, 90, 66, 58, 45, 38, 93, 77, 50]. Classify each score into a letter grade based on the following rules:

- A: 80 and above
- B: 60 to 79
- C: 40 to 59
- F: below 40

1. Use `if-elif-else` statements to classify each score into its grade.
2. Count how many students fall into each grade.



3. Compute the probability of getting each grade, using:

$$P(\text{Grade}) = \frac{\text{Count of Grade}}{\text{Total Scores}}$$

4. Print the probabilities in this format:

P(A) = 0.2

P(B) = 0.3

P(C) = 0.3

P(F) = 0.2

## 2.5. Count Failures or Absentees

You are given a list of student records where each element is a tuple of the form (**status**, **marks**): [('pass', 60), ('fail', 30), ('absent', 0), ('pass', 85), ...]. Write a program to:

- Count how many students either failed or were absent
- Compute the empirical probability:

$$P(\text{fail or absent}) = \frac{\text{fail or absent}}{\text{total students}}$$

- Use the **or** operator.

## 2.6. Identify Invalid or High Values

You are given a list of time measurements in seconds: [10, 12, 15, -1, 0, 20, 18, 25, -5, 30]. Write a program to

- Count how many values are not valid (zero or negative)
- Count how many values are valid and greater than 15
- Use the **not** and **and** operators appropriately.

## 2.7. Rainfall Categorization Using Combined Conditions

Given rainfall data (in mm): [2, 0, 5, 30, 45, 10, 3, 25, 0, 50], write a program to classify each value as

- “None” if value is 0
- “Low” if value  $> 0$  and  $\leq 10$
- “Moderate” if value  $> 10$  and  $\leq 30$
- “Heavy” if value  $> 30$

**Instructions:** Use **if-elif-else** with combined **and** conditions.

## 2.8. Estimate Conditional Probability

Given a list of session data represented as pairs [(‘user1’, 1), (‘user2’, 0), (‘user3’, 1), ...], where 1 indicates a purchase and 0 indicates no purchase. Assume only users who visited the site between 6pm and 9pm are labeled. Estimate

$$P(\text{purchase} \mid \text{visited between 6pm and 9pm}) = \frac{\text{No of purchases}}{\text{No of total sessions}}$$

Count purchases manually using a loop. Use only string labels and integers.

## 2.9. Empirical Distribution Function (PMF)

Given a list of integers representing number of customer support tickets per day for 15 days: [2, 3, 2, 4, 3, 3, 1, 2, 4, 4, 2, 2, 1, 3, 4], compute the empirical PMF:

$$P(X = k) = \frac{\text{Number of times } k \text{ appears}}{\text{Total observations}}, \quad \text{for } k \in \{1, 2, 3, 4\}$$

Use a loop to count how many times each value appears.

## 2.10. Empirical Cumulative Distribution Function (CDF) Using Lists

You are given a list containing the ages of visitors to a website (in years):

[19, 22, 25, 30, 21, 24, 20, 18, 28, 27]

1. Sort the list in ascending order using the `sort()` function.
2. For each value  $x$  in the sorted list, compute the empirical cumulative distribution function (CDF) using

$$P(X \leq x) = \frac{\text{Number of values } \leq x}{\text{Total number of values}}$$

3. Store the results in a list of tuples of the form  $(x, P(X \leq x))$ .
4. Print all values in a tabular format showing the age and the cumulative probability.

**Hint:** You may use a `for` loop and `append()` to build the list of tuples. Use `len()` to get the total number of values.

## 2.11. Function to Run a Monte Carlo Experiment

You want to estimate the value of  $\pi$  using random sampling. Write a Python function `def estimate_pi(trials):` to

1. Randomly generate points in the unit square:  $x, y \in [0, 1]$
2. Count how many of those points fall inside the quarter unit circle, i.e., satisfy

$$x^2 + y^2 \leq 1$$

3. Classify each point as ‘Inside the quarter circle’, if  $x^2 + y^2 \leq 1$  and ‘Outside the quarter circle’, otherwise.

4. Use the following formula to return the estimated value of  $\pi$

$$\hat{\pi} = 4 \times \frac{\text{points inside circle}}{\text{total trials}}$$

5. Plot the sampling points using `matplotlib` library with points inside the circle in blue and points outside the circle in red. Include a legend and an appropriate title such as ‘‘Monte Carlo Simulation of  $\pi$ ’’.

**Learning Outcome** Through this exercise, students can learn basic Python syntax and fundamental concepts in probability and statistics.

© 2025 Rekha R. CE, FoT, UoD

### 3. Randomized Test Generation from Question Bank

A teacher maintains a question bank containing 100 questions labeled Q1 to Q100 (you can create a NumPy array of 100 questions labeled Q1 to Q100 for convenience in random sampling later). For each unit test, 5 questions are selected at random from the bank. This selection may be done with or without replacement. (From now on, you are free to use suitable libraries to implement the solutions.)

#### 1. *Simulate Question Selection*

- (a) Write a function to randomly select 5 questions **with replacement** from the bank.
- (b) Write a function to randomly select 5 questions **without replacement**.

#### 2. *Run Simulations*

- (a) Generate 10,000 simulated tests (each with 5 questions).
- (b) Count how many times each question is selected in both modes.

#### 3. *Compute Expected Unique Questions*

- (a) For sampling **with replacement**, compute the average number of *unique questions* in each test across simulations. Compare the theoretical and simulated results.
- (b) For sampling **without replacement**, verify that the number of unique questions is always 5. Compare the theoretical and simulated results.

#### 4. *Simulate Conditional Probability*: Given Q1 is selected, what is the chance Q2 is also selected? Compare the theoretical and simulated results.

5. Let us assume that 30 questions are randomly labeled as *easy* and the remaining 70 questions are labeled as *hard*. During a test, one question is randomly selected (with replacement), and a student attempts to answer it. You are told that the student answered the question *correctly*. Your goal is to estimate the probability that the question was an easy one. In probability terms, you are estimating

$$P(\text{Easy} \mid \text{Correct})$$

- If a question is labeled **easy**, the student answers it correctly with probability  $P(C \mid E) = 0.9$
- If a question is labeled **hard**, the student answers it correctly with probability  $P(C \mid H) = 0.4$

Use Bayes' Theorem to compare your simulated result with the theoretical value. Write down your observation in the Markdown cell.

6. Group 1,000 tests and record the average number of distinct questions. Repeat this for 10 such groups. Create a boxplot with 10 boxes, where each box shows the average number of distinct questions.

7. In 10,000 tests (5 questions each), count how often each pair of questions is selected together. Plot a  $100 \times 100$  heatmap of co-selection frequencies.

8. *Biased Sampling*

- (a) Modify the selection method so that questions Q1 to Q20 are more likely to be selected.
- (b) Plot the frequency histogram and note down the impact of bias in a Markdown cell.

**Learning Outcome** Through this exercise, students will understand key concepts in probability and statistics through simulation-based exploration. They will compare random sampling methods, estimate probabilities empirically, and apply conditional probability and Bayes' Theorem in realistic scenarios. Through data visualization and analysis, students will examine distribution patterns, co-occurrences, and the impact of biased sampling.

---

### 3.1. Solution

Create an array of 100 items where each item is a question labeled Qi.

1. *Simulate Question Selection*

- (a) **With Replacement:** To select 5 questions, pick one question at a time and repeat five times. After a question is picked, it is “put back” into the pool. Each draw is independent and every question has equal probability each time. The same question may appear multiple times in the selection (e.g., Q17, Q42, Q3, Q89, Q42).
- (b) **Without Replacement:** Once a question is picked, it is removed from the pool for that test. Each draw is dependent, since the pool shrinks. The 5 questions will always be distinct.

In NumPy use `numpy.random.choice()` with the parameter `replace=True` (for with replacement) or `replace=False` (for without replacement). Input is the array of questions [Q1, Q2, ..., Q100] and the desired sample size (5).

In Jupyter Notebook, NumPy can be imported in the standard way, using an alias (e.g., `np`). This allows access to functions such as `np.array()` or `np.random.choice()`. You may encounter an error that NumPy is “not recognized” in Jupyter. This usually occurs when NumPy is not installed in the Python environment that Jupyter is currently using. To resolve this ensure that NumPy is installed in the same environment as Jupyter. Restart the Jupyter kernel after installation so that changes are applied.

2. *Run Simulations*

- (a) Assume you have already written a function that produces one test by randomly selecting 5 questions from the bank (with replacement and without replacement). Now, write another function that calls it repeatedly 10,000 times.

- (b) For each test, count how many times each selected question appears. You may redirect the output to a file.

### 3. Compute Expected Unique Questions

- (a) *With replacement*: A question may appear multiple times in the same test (so frequencies can be 2, 3, or more). You may also keep a count of unique questions in a test. Find the average value across the simulations.

*Theoretical analysis*: Let there be a question bank of size  $N$  and a test of size  $k$  with replacement. Let  $U$  be the number of unique questions in the test. Define indicator variables

$$I_i = \begin{cases} 1 & \text{if question } i \text{ appears at least once in the test,} \\ 0 & \text{otherwise,} \end{cases}$$

so that

$$U = \sum_{i=1}^N I_i$$

By linearity of expectation,

$$E[U] = \sum_{i=1}^N E[I_i]$$

Now,

$$E[I_i] = P(I_i = 1) = 1 - P(\text{question } i \text{ never appears}).$$

The probability that question  $i$  does not appear in a single draw is

$$1 - \frac{1}{N}$$

so the probability it never appears in  $k$  independent draws is

$$\left(1 - \frac{1}{N}\right)^k$$

Hence,

$$E[I_i] = 1 - \left(1 - \frac{1}{N}\right)^k$$

Therefore, the expected number of unique questions is

$$E[U] = N \left[ 1 - \left(1 - \frac{1}{N}\right)^k \right]$$

You may implement  $E[U]$  for generic  $N$  and  $k$ . You can supply  $N = 100$  and  $k = 5$  for the question.

- (b) *Without replacement*: Every chosen question should appear exactly once (so frequencies are always 1). You may also keep a count of unique questions which should be 5 in a test. Find the average value across the simulations.

$$E[U] = 5$$

4. *Simulate conditional probability*

- Theoretical analysis: Let

$$A = \{\text{Q1 appears at least once}\}, \quad B = \{\text{Q2 appears at least once}\}.$$

We want to compute

$$P(B | A).$$

- 1) With replacement:

$$P(B | A) = \frac{P(A \cap B)}{P(A)}.$$

Using inclusion-exclusion

$$P(A) = 1 - \left(1 - \frac{1}{100}\right)^5, \quad P(A \cap B) = 1 - 2\left(1 - \frac{1}{100}\right)^5 + \left(1 - \frac{2}{100}\right)^5.$$

Therefore,

$$P(B | A) \approx 0.0396 \quad (\text{about } 3.96\%).$$

You may implement  $P(B | A)$  for generic  $N$  and  $k$ . You can supply  $N = 100$  and  $k = 5$  for the question.

- 2) Without replacement:

Given Q1 is in the set, the remaining 4 slots are chosen from the other 99 questions. The chance that Q2 is among those 4 is

$$P(B | A) = \frac{4}{99} \approx 0.0404 \quad (\text{about } 4.04\%).$$

- Simulation result:

- (a) Count the number of tests,  $n_{Q1}$ , containing Q1.
- (b) Count the number of tests,  $n_{Q1, Q2}$ , containing both Q1 and Q2.
- (c) Compute the conditional probability:

$$P(Q2 | Q1) = \frac{n_{Q1, Q2}}{n_{Q1}}$$

- (d) Note: If  $n_{Q1} = 0$ , the conditional probability is undefined.

5. *Easy/Hard*

- Since the questions are not pre-labeled, you must label them yourself. The question suggests that the labeling is random. There are two ways to do that.
  - Option A.
    - \* Randomly choose 30 distinct questions.
    - \* Mark those 30 as Easy, mark the other 70 as Hard.
 This produces a fixed labeled bank of 100 questions with exactly 30 easy ones.
  - Option B.

- \* For each question  $i$  generate a uniform random number  $u_i \in [0, 1]$ .
- \* If  $u_i < 0.3$  mark question  $i$  as Easy; otherwise mark it Hard.

Ensure you have labeled only 30 questions as easy.

- Theoretical analysis: By Bayes' theorem,

$$P(E | C) = \frac{P(C | E) P(E)}{P(C)},$$

where

$$P(C) = P(C | E)P(E) + P(C | H)P(H).$$

- Simulation result: After labeling the difficulty level simulate correct answers for a test:
  - (a) Generate a random number  $r \in [0, 1]$ .
  - (b) If the question is labeled *Easy* and
 

if  $r < 0.9$ , mark it Correct; otherwise, Incorrect.
  - (c) If the question is labeled *Hard* and
 

if  $r < 0.4$ , mark it Correct; otherwise, Incorrect.
  - (d) Count the number of Correct Easy questions and total Correct questions for this test.
  - (e) Repeat the above simulation for  $N = 10,000$  tests.
  - (f) Compute the empirical probability:

$$\hat{P}(E | C) = \frac{\text{total Correct Easy questions over all simulations}}{\text{total Correct questions over all simulations}}.$$

## 6. Box plot

- (a) For each test count the number of unique questions used.
- (b) Repeat this for  $N = 1,000$  simulations.
- (c) Find the average number of unique questions in this group.
- (d) Repeat the above for 10 groups of simulations.
- (e) Visualize the variation using a box-plot

## 7. Heatmap

- (a) For each test, list all  $\binom{5}{2} = 10$  pairs of questions.
- (b) Maintain a  $100 \times 100$  matrix  $M$ , where

$$M[i, j] = \text{number of times question } i \text{ and } j \text{ were selected together.}$$

and  $M[i, i] = 0$ .

- (c) Repeat the above for all tests.
- (d) Plot  $M$  as a heatmap



- Axes: question indices  $1, \dots, 100$ .
- Color intensity: co-selection frequency.

8. *Biased sampling*

(a) *Modify selection method*

- For questions 1 to 20, assign a higher weight. These are “more likely” to appear in a test.
- For questions 21 to 100, assign a lower weight. These are “less likely” to appear in a test.
- Normalize the weights to make them valid probabilities by dividing each weight by the total sum of all weights.
- Run 10,000 simulations of tests with 5 questions each.

(b) *Frequency histogram*

- Across all simulations, count how many times each question was selected.
- X-axis shows the question number and Y-axis shows the number of times the question was selected across all simulations.
- You may color the first 20 bars differently to emphasize the “biased” set.

© 2025 Rekha R. CE, FoT, UoD

## 4. Bike Rental Forecasting with Discrete and Continuous Random Variables

An urban bike-sharing company wants to understand and forecast the rental patterns of bikes across the day. Rentals are affected by weather and the duration of use varies with user type and time. It logs two types of data each day

1. the number of bikes rented each hour (count data) and
2. the duration of each ride in minutes (time data).

### Tasks:

#### 1. Data generation:

- (a) Generate an array of size 24 (for 24 hours), where each value represents the number of rentals in that hour. Use any reasonable whole numbers (e.g., between 0 and 15).
- (b) For each rental, generate a random ride duration in minutes in the range 5 to 60.

Generate the dataset in *three* different ways:

- (a) *Random assignment*: Use random integers within the range (e.g., rentals between 0–15 per hour, ride durations between 5–60 minutes).
- (b) *Statistical simulation*: Use probability distributions (e.g., Poisson for rentals and Exponential for ride durations).

- *Modeling rentals with Poisson distribution*. We have seen in our lectures that the Poisson distribution models the number of events in a fixed time interval, given an average rate  $\mu$ . However, a single  $\mu$  for all hours assumes a constant average rate, which does not capture rush hours. To simulate rush hours, we can assign different  $\mu$  values to different periods of the day. For example

- Morning peak (7–9 am):  $\mu_{pm} = 12$
- Midday (10 am–3 pm):  $\mu_{md} = 5$
- Evening peak (5–8 pm):  $\mu_{pe} = 15$
- Night (after 9 pm):  $\mu_{ng} = 2$

We then generate the number of events for each period separately using the Poisson distribution

$$X_{per} \sim \text{Poisson}(\mu_{per})$$

where  $X_{per}$  is the number of rentals in the period  $per$ , and  $\mu_h$  is the corresponding rate.

*Implementation idea:*

- i. For now you can hard code  $\mu$  for each time period.
- ii. For each period, generate a Poisson random variable with the corresponding  $\mu$ .

iii. Combine the results to get a full day simulation.

This approach allows the simulation to reflect variable intensity of events throughout the day.

- *Modeling ride durations with an exponential distribution.* The *exponential distribution* models the time until an event occurs. For ride durations, the event is the ride ending. Let  $T_i$  be the duration of ride  $i$ :

$$T_i \sim \text{Exponential}(\text{scale} = \text{average duration})$$

This distribution is suitable because the durations are always positive, and in high traffic regions, most rides are short with a long tail of longer rides. It is memoryless in that the probability of a ride ending in the next minute does not depend on how long it has already lasted. Also most rides are around the average duration, but some are shorter or longer, giving realistic variation. You can use the `random.exponential(scale=..., size=...)` function in the `numpy` library for exponential distribution where `scale` is an average ride duration of your choice and `size` is the total rentals computed above.

- (c) *Manual entry.* Read values from a given CSV file entered manually (based on realistic assumptions).

*Note: You do not need to solve the later tasks on all three datasets. The above is to help you understand different ways of creating data.*

## 2. Probability estimation:

- (a) Use a generated dataset of your choice to estimate the probability that more than 10 bikes are rented in a randomly chosen hour over 10,000 days.
- (b) Find the hour with the maximum rentals in the first day.
- (c) What is the empirical probability of observing that maximum value over 10,000 days?
- (d) Empirically estimate the probability that a randomly selected ride lasted more than 30 minutes over 10,000 days. (*Hint*: Pool together all the ride durations across the 10,000 days.)
- (e) Empirically estimate the probability that a ride lasted between 20 and 25 minutes over 10,000 days. (*Hint*: Pool together all the rides of durations between 20 and 25 minutes across 10,000 days.)

## 3. Metrics:

- (a) Compute empirical mean and variance of the ride durations over 10,000 days.

## 4. Visualization:

- (a) Create a bar chart showing the number of rentals for each of the 24 hours of the first day. Color the bars differently (e.g., in red) where rentals exceeded 10 in an hour.
- (b) Create a bar chart showing the average number of rentals across all days. Color the bars differently (e.g., in red) where rentals exceeded 10 in an hour.

- (c) Plot the same data as a line graph to show rental trend over a randomly chosen day.
- (d) Plot the frequency of each unique rental count (e.g., how often 0 bikes were rented, 1 bike, ..., 15 bikes) on a randomly chosen day.
- (e) Plot a histogram of all ride durations using `hist()` function of the `matplotlib`'s `pyplot` module with 20–30 bins. Add labels and a vertical line at the mean.
- (f) Plot a smoothed version of the histogram using `matplotlib`.
- (g) Overlay a horizontal line or shaded region to highlight rides lasting more than 30 minutes.

**Learning Outcome** Through this exercise, students will explore the behavior of discrete and continuous random variables. Students will gain hands-on experience in data generation in a realistic setting, probability estimation, data visualization, and statistical interpretation.

---

#### 4.1. Solution

##### 2. Probability estimation

- (a) Create a  $10000 \times 24$  matrix where each entry is the number of rentals in that day-hour. You may treat every hour equally.  $N = 10000 * 24$  sampled hours. Count how many of those entries are  $> 10$  and divide by  $N$ . That fraction is the estimated probability.
- (c) For each simulated day, compute the daily maximum  $M_d$ . To estimate the probability that the daily maximum equals a specific value  $m$

$$P(M = m) = \frac{\text{count } M_d = m}{10000}$$

## 5. Website Session Analysis using Binomial and Normal Distributions

The goal is to analyze user activity on an e-commerce website. Download and use Kaggle dataset “anthonytherrien/website-traffic” for the problem. You have to model and simulate this behavior using probability distributions to understand purchasing patterns and advise the website designer.

### Tasks: Binomial Distribution

1. Assume that each session has an independent probability  $p = 0.2$  of resulting in a purchase. For a random sample of 50 sessions, compute the probability that exactly 10 sessions result in a conversion.
2. Out of 50 sessions from Organic traffic, what is the probability that at least 25 sessions result in conversions? Assume  $p = 0.2$  for each session.
3. From the dataset, identify the most engaged 10% of visitors (those with the lowest bounce rates) and report what percentage of the total sessions do they represent.
4. Compute the conversion rate within most engaged group. Compare it with the conversion rate of the remaining 90% of visitors.
5. Out of 200 sessions, use the Normal approximation to the Binomial to estimate the probability that more than 50 sessions result in conversions, given  $p = 0.2$ .

### Tasks: Normal Distribution

1. Suppose session durations (in minutes) are approximately normally distributed with mean  $\mu = 5$  and standard deviation  $\sigma = 2$ . Compute the probability that a randomly selected session lasts more than 8 minutes.
2. Assume time on page is normally distributed with mean  $\mu = 3$  minutes and standard deviation  $\sigma = 1.2$  minutes. Find the probability that a visitor spends between 2 and 4.5 minutes on a page.
3. Suppose the number of pages viewed per session follows a Normal distribution with mean  $\mu = 4$  and standard deviation  $\sigma = 1.5$ . For a random sample of 30 sessions, find the probability that the average number of pages viewed exceeds 5.
4. Compare session durations for Organic vs Paid traffic. Assume Organic has  $\mu_1 = 5.2$ ,  $\sigma_1 = 1.8$  and Paid has  $\mu_2 = 4.7$ ,  $\sigma_2 = 2$ . Compute the probability that a randomly chosen Organic session lasts longer than a randomly chosen Paid session. (Assume independence.)
5. Assume that previous visits per visitor follow a Normal distribution with mean  $\mu = 3$  and standard deviation  $\sigma = 1.5$ . Find the probability that a randomly selected visitor has at least 5 previous visits.
6. Traffic sources generate sessions with session duration approximately Normal with mean  $\mu = 5$  and standard deviation  $\sigma = 2$ . If we sample 40 sessions, compute the probability that the sample mean duration is less than 4.5 minutes.

**Learning Outcome** Through this exercise, students could retrieve data from publicly available datasets, deepen their understanding of distribution-based modeling, compute relevant probabilities, and compare theoretical distributions with empirical data.

---

### 5.1. Solution

**Tasks:** Binomial Distribution

#### 1. Probability of exactly 10 purchases

- *Theoretical analysis.* Model this problem as

$$X \sim \text{Bin}(n = 100, p = 0.2)$$

Find the probability for exactly 10 purchases.

- *Empirical estimate*

(a) Convert conversion rate to a binary variable as follows

$$\text{purchase} = \begin{cases} 1 & \text{if conversion rate} \geq 0.5 \\ 0 & \text{if conversion rate} < 0.5 \end{cases}$$

Now, each session is either a *success* (purchase occurred) or *failure* (no purchase).

- (b) Randomly create  $n$  groups of 50 sessions each. (You may hardcode  $n$ ).
- (c) For each group, count how many purchases occurred.
- (d) For  $k = 10$ , the empirical probability is given by

$$\hat{P}(X = 10) = \frac{\text{Number of groups with exactly 10 purchases}}{n}$$

#### 2. Probability of atleast 25 conversions

- *Theoretical analysis.*

$$P(X \geq 25) = P(X = 25) + P(X = 26) + \dots + P(X = 50)$$

Using the Binomial formula, each term is

$$P(X = k) = \binom{50}{k} p^k (1 - p)^{50-k}, \quad k = 25, 26, \dots, 50$$

- *Empirical estimate.* The idea is similar to the one in the previous question.

#### 3. Engaged visitors

- (a) Compute engagement rate for each session

$$\text{engagement rate} = 1 - \text{bounce rate}$$

- (b) Sort all sessions in the dataset by their engagement rate in descending order.

- (c) Pick the sessions that cover the top 10% of visitors.
- (d) % of total sessions with most engaged visitors

$$\frac{\text{number of most engaged sessions}}{\text{number of sessions}}$$

4. *Conversion.* Let the conversion rate within most engaged group, be denoted  $\hat{C}R_{\text{engaged}}$  and the conversion rate of the remaining 90% of visitors be denoted  $\hat{C}R_{\text{rest}}$ .

- (a) Find the average conversion rate for the most engaged group.
- (b) Compute the average conversion rate for the remaining sessions. In which category your result is:
  - If  $\hat{C}R_{\text{engaged}} > \hat{C}R_{\text{rest}}$ , we can conclude \_\_\_\_\_.
  - If  $\hat{C}R_{\text{engaged}} \approx \hat{C}R_{\text{rest}}$ , it suggests \_\_\_\_\_.
  - If  $\hat{C}R_{\text{engaged}} < \hat{C}R_{\text{rest}}$ , it indicates \_\_\_\_\_.

5. Out of 200 sessions, use the Normal approximation to the Binomial to estimate the theoretical probability that more than 50 sessions result in conversions, given  $p = 0.2$ .

Let  $X \sim \text{Binomial}(n = 200, p = 0.2)$ . For large  $n$ ,  $X$  can be approximated by a Normal distribution with

$$\mu = np = 200 \cdot 0.2 = 40, \quad \sigma = \sqrt{np(1-p)} = \sqrt{200 \cdot 0.2 \cdot 0.8} \approx 5.657$$

We are interested in the probability that more than 50 sessions result in conversions  $P(X > 50) = P(X \geq 51)$ . We approximate

$$P(X \geq 51) \approx P\left(Z > \frac{50.5 - \mu}{\sigma}\right) = P\left(Z > \frac{50.5 - 40}{5.657}\right) = P(Z > 1.856)$$

You may use `scipy` library to compute  $P(Z > 1.856)$ .

### Tasks: Normal Distribution

1. Suppose session durations (in minutes) are approximately normally distributed with mean  $\mu = 5$  and standard deviation  $\sigma = 2$ . Compute the probability that a randomly selected session lasts more than 8 minutes.
2. Assume time on page is normally distributed with mean  $\mu = 3$  minutes and standard deviation  $\sigma = 1.2$  minutes. Find the probability that a visitor spends between 2 and 4.5 minutes on a page.
3. Suppose the number of pages viewed per session follows a Normal distribution with mean  $\mu = 4$  and standard deviation  $\sigma = 1.5$ . For a random sample of 30 sessions, find the probability that the average number of pages viewed exceeds 5.
4. Compare session durations for Organic vs Paid traffic. Assume Organic has  $\mu_1 = 5.2$ ,  $\sigma_1 = 1.8$  and Paid has  $\mu_2 = 4.7$ ,  $\sigma_2 = 2$ . Compute the probability that a randomly chosen Organic session lasts longer than a randomly chosen Paid session. (Assume independence.)

5. Assume that previous visits per visitor follow a Normal distribution with mean  $\mu = 3$  and standard deviation  $\sigma = 1.5$ . Find the probability that a randomly selected visitor has at least 5 previous visits.
6. Traffic sources generate sessions with session duration approximately Normal with mean  $\mu = 5$  and standard deviation  $\sigma = 2$ . If we sample 40 sessions, compute the probability that the sample mean duration is less than 4.5 minutes.

© 2025 Rekha R. CE, FoT, UoD



## 6. Analysis of Medical Data through the Lens of Bivariate Normal Distribution

In real-world medical datasets, diagnostic features such as glucose levels, BMI, and age often exhibit relationships that can be modeled using joint probability distributions. By analyzing covariance, correlation, and simulated data, we can better understand how pairs of diagnostic features interact and how these relationships may influence health outcomes. To begin your analysis you can download the Pima Indians Diabetes dataset from Kaggle available at <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>.

### Tasks:

1. *Data Cleaning and Processing.* The dataset contains diagnostic measurements, but some columns such as contain many zeros, which may be treated as missing values. Write Python code to detect these anomalies and replace the missing values using the realistic mean imputation

$$\text{Imputed Value} = \mu + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_{\text{small}}^2)$$

where  $\mu$  is the mean of the data,  $\epsilon$  is the random noise generated from a normal distribution with a small hard coded variance  $\sigma_{\text{small}}^2$ .

2. *Bivariate Normal Distribution Analysis.* For the features **Glucose** and **BMI** features from the dataset

- (a) Estimate the mean vector and covariance matrix of the selected features from the dataset.

$$\text{covariance matrix} = \begin{bmatrix} \text{Var}(X) & \text{Cov}(X, Y) \\ \text{Cov}(Y, X) & \text{Var}(Y) \end{bmatrix}$$

- (b) Plot a scatter plot of the two features and overlay the contours of the fitted bivariate normal distribution.
- (c) Generate a 3D surface plot (bell-shaped) representing the joint probability density of the bivariate normal distribution. Label the following based on the data:

- Peak of the bell: highest  $f(x, y)$  value, located at the mean vector  $(\mu_X, \mu_Y)$ .
- Spread along  $X$  and  $Y$ : edges of the bell at  $\mu_X \pm \sigma_X$  and  $\mu_Y \pm \sigma_Y$ , reflecting the variance of each feature.
- Tilt: determined from the correlation coefficient; a small arrow along the major and the minor axis of the tilt.

3. *Conditional Probability of Diabetes.* Assume Glucose and BMI jointly follow a bivariate normal.

- (a) Compute  $P(\text{Outcome} = 1 \mid \text{Glucose} = x, \text{BMI} = y)$  using conditional distributions and Gaussian likelihood. Through this estimate individualized diabetes risk.
- (b) Generate a 2D grid of Glucose and BMI values covering the range of the dataset. Plot the results using

- a heatmap showing the probability of being diabetic for each  $(x, y)$  combination. Use red colour to indicate high risk and blue for low risk.
- Contour lines for fixed probability levels of 0.2, 0.5, and 0.8.

**Learning Outcome** Upon completing this section, students will be able to clean and preprocess datasets, model pairs of diagnostic features using bivariate normal distributions, visualize joint probability densities and correlation structures, and estimate individualized risk probabilities. They will also be able to interpret scatter plots, contour plots, 3D surface plots, and heatmaps in the context of medical risk assessment.

---

## 6.1. Solution

1. *Data Cleaning and Processing.* To impute missing values using a stochastic approach

- (a) Decide on a small standard deviation,  $\sigma_{\text{small}}$  depending on the feature scale.
- (b) For each missing value, draw a random number  $\epsilon$  from a normal distribution

$$\epsilon \sim \mathcal{N}(0, \sigma_{\text{small}}^2)$$

- (c) Add this random number to the mean of the observed data to create the imputed value

$$\text{Imputed Value} = \mu + \epsilon$$

where  $\mu$  is the mean of the observed (non-missing) values.

This way, each missing value gets a meaningful value which preserves variability in the data. *Note:* You must be careful about the imputed value generated. The imputed value should be meaningful for the feature.

2. *Bivariate Normal Distribution Analysis.*

- (a) Compute the mean vector  $\mu = [\mu_X, \mu_Y]$ , where each component is the average of the respective feature

Compute the covariance matrix  $\Sigma$ , which captures the variance of each feature and the covariance between them

$$\Sigma = \begin{bmatrix} \text{Var}(X) & \text{Cov}(X, Y) \\ \text{Cov}(Y, X) & \text{Var}(Y) \end{bmatrix}$$

- (b) To visualize the joint distribution of two selected features you may use `matplotlib.pyplot` for plotting the scatter plot and contour lines, and `scipy.stats.multivariate_normal` to compute the bivariate normal probability density.
- (c) Use `matplotlib.pyplot` for plotting the 3D surface using `Axes3D`. Compute the tilt direction from the data and draw a small arrow on the 3D bell plot along this direction using `ax.quiver()`.
  - Use `ax.scatter()` to place a visible point at the peak  $(\mu_X, \mu_Y, f(x, y)_{\text{peak}})$ . Label it as “Mean vector” using `ax.text()`.

- Draw points at  $(\mu_X \pm \sigma_X, \mu_Y, f(x, y))$  and  $(\mu_X, \mu_Y \pm \sigma_Y, f(x, y))$  to indicate edges of the spread and label them appropriately.
- To indicate the direction of correlation (tilt) on the 3D bell-shaped plot, follow these steps:
  - i. Identify the starting point. Let  $(X_0, Y_0, Z_0) = (\mu_X, \mu_Y, f(x, y)_{\text{peak}})$ .
  - ii. Compute the direction vector  $(U, V, W)$ 
    - Compute the covariance matrix of the two features.

$$\Sigma = \begin{bmatrix} \text{Var}(X) & \text{Cov}(X, Y) \\ \text{Cov}(Y, X) & \text{Var}(Y) \end{bmatrix}$$

- Find the eigenvectors and eigenvalues of the covariance matrix using the function `numpy.linalg.eig(cov_matrix)`.

```
eigvals, eigvecs = np.linalg.eig(cov_matrix)
```

`eigvals` is a 2 element array of eigenvalues. The larger eigenvalue ( $\lambda_{\max}$ ) corresponds to the major axis variance and the smaller eigenvalue ( $\lambda_{\min}$ ) corresponds to the minor axis variance. `eigvecs` is  $2 \times 2$  matrix of eigenvectors. The column corresponding to  $\lambda_{\max}$  gives the direction of the major axis and the column corresponding to  $\lambda_{\min}$  gives the direction of the minor axis.

```
largest_index = np.argmax(eigvals)
smallest_index = np.argmin(eigvals)
largest_eigvec = eigvecs[:, largest_index]
smallest_eigvec = eigvecs[:, smallest_index]
```

- Set first and second values in the `largest_eigvec` as  $U$  and  $V$ , respectively.
- Set  $W = 0$  if the arrow should lie horizontal;  $W \neq 0$  if you want it slightly inclined along the surface.
- Draw the arrow for major axis using `ax.quiver(X_0, Y_0, Z_0, U, V, W)`.
- You may do the same to draw the minor axis.

### 3. Conditional Probability of Diabetes.

- (a) Split the dataset by “Outcome”. Let Group 0 be non-diabetic patients and Group 1 be diabetic patients. For each group, compute mean vector for (Glucose, BMI) and covariance matrix  $\Sigma$ . This gives two bivariate normal distributions: one for diabetics and one for non-diabetics.
- (b) Compute the bivariate normal probability density (PDF) under each group

$$f_0(x, y) = \text{pdf for non-diabetics}$$

$$f_1(x, y) = \text{pdf for diabetics}$$

These are called the likelihoods of observing the patient’s features in each group.

- (c) Use the prior probabilities of each group from the dataset

$$P(\text{Outcome} = 0) = \frac{\text{\#non-diabetics}}{\text{total patients}}, \quad P(\text{Outcome} = 1) = \frac{\text{\#diabetics}}{\text{total patients}}$$

- (d) Compute the posterior probability that the patient is diabetic

$$P(\text{Outcome} = 1 \mid x, y) = \frac{f_1(x, y) \cdot P(\text{Outcome} = 1)}{f_0(x, y) \cdot P(\text{Outcome} = 0) + f_1(x, y) \cdot P(\text{Outcome} = 1)}$$

Use `scipy.stats.multivariate_normal` to compute  $f_0(x, y)$  and  $f_1(x, y)$ .

© 2025 Rekha R. CE, FoT, UoD

## 7. Understanding Capital-Gain Distributions Through the Central Limit Theorem (CLT)

Assume that you are a data analyst at a financial consultancy. The firm is investigating whether **capital-gain** can be used as a reliable predictor of income levels (i.e., whether an individual earns  $>50K$  or  $\leq 50K$ ). However, this variable is known to be heavily skewed. Your task is to use the Central Limit Theorem (CLT) to analyze how reliable the sample mean of **capital-gain** is across different demographic groups, specifically those grouped by education level.

### Tasks

1. Download and use the Kaggle dataset from `uciml/adult-census-income`. Clean the dataset and use only the following columns: **capital-gain**, **education**, **income**, and **hours-per-week**. Create subgroups based on education levels (e.g., Bachelors, HS-grad, Masters, etc.).
2. For each education subgroup, plot a histogram of the raw **capital-gain** values. Make a note of the share of the plot.
3. For each subgroup based on education level
  - (a) Draw 1000 samples of sizes  $n = 5$ ,  $n = 30$ , and  $n = 100$  from the **capital-gain** column with replacement.
  - (b) Compute the mean of each sample.
  - (c) Plot histograms of sample means and overlay them with normal probability density functions (PDFs) using sample mean and sample sd.
  - (d) Compare how the distribution of sample means changes with increasing sample size.

**Learning Outcome** Through this exercise, students could reinforce their understanding of sampling variability, distributional properties, and inference-making using the CLT, especially when dealing with non-normally distributed populations. Even though the **capital-gain** variable need not be normal, the distribution of its sample means tends to become normal as  $n$  increases which is precisely what the Central Limit Theorem demonstrates.

## 8. Confidence Interval Estimation for Mean Rainfall with Visualization

You are part of a climate research unit analyzing daily rainfall data (in millimeters) for a particular region. Your objective is to estimate the true average daily rainfall using a confidence interval and visualize the statistical conclusions.

### Tasks

1. Simulate a sample of daily rainfall for 50 days using a normal distribution. Assume the population mean rainfall is around 10 mm/day with a population standard deviation of 3 mm.
2. Compute the sample mean  $\bar{x}$  and construct a 95% confidence interval for the true mean using the formula

$$\bar{x} \pm z_{\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}, \quad \text{where } z_{\alpha/2} = 1.96$$

Do not use any library for these.

3. Plot a histogram of the rainfall sample using `matplotlib`. Annotate the plot with
  - A vertical line at the sample mean
  - Vertical lines showing the lower and upper bounds of the confidence interval
  - A legend describing each line
4. Simulate 100 samples of size 50 and compute the 95% confidence interval for each. For each interval, record whether it contains the true mean of 10 mm.
5. Plot all 100 confidence intervals as horizontal lines in a graph
  - Color the intervals that contain the true mean differently from those that do not.
  - Include a horizontal reference line at  $y = 10$  to show the true mean.

**Learning Outcome** Through this exercise, students will learn to estimate a population mean using confidence intervals and reinforce their understanding through visualization. They will also explore the interpretation of confidence intervals using simulation and graphical representation.

## 9. Z-Test for Average Food Delivery Time Using a Sample of 100

Assume that you are a data analyst at a food delivery company evaluating whether delivery performance meets the company's promise. The company advertises an average delivery time of 30 minutes. From prior analysis, the population standard deviation is known to be  $\sigma = 8$  minutes. You are provided the Kaggle dataset "gauravmalik26/food-delivery-dataset/train.csv", which contains a large number of delivery records, including the time column. Download and clean the dataset for use in this exercise.

### Tasks

1. Randomly select a sample of 100 delivery times from the dataset.
2. Based on your sampled data, investigate whether recent delivery performance aligns with the advertised promise. Perform a one-sample Z-test. Note down the hypotheses in Markdown cell.
3. Compute the  $p$ -value using `scipy.stats.norm.sf(z)`.
4. What does your analysis imply for the company's delivery time claim at a 5% tolerance level? Report your finding in a Markdown cell and support it with appropriate statistical evidence.
5. As a data analyst, you are expected to detect patterns in operational behavior. Use the available information to examine if delivery duration seems to be influenced by the delivery agent's age. Present your analysis visually as a scatter plot with delivery person age on the X-axis and delivery time minutes in the Y-axis. Comment on any notable patterns (e.g., younger or older delivery persons taking less/more time), clusters, or deviations you observe.
6. The operations team wants to better allocate drivers during rush hours. Use the available traffic condition data to explore how road density levels affect delivery times. Present your insights in a compact visual summary. For this, you may create a boxplot showing delivery time minutes for different values of Traffic Density: Low, Medium, High, Jam. Each box should represent delivery times under a specific density level. Make an operational recommendation based on your findings on how road conditions impact delivery performance.

Steps to draw a Boxplot

- (a) Group delivery times by traffic density. Example: Low - [25, 22, ...] Medium - [32, 29, ...], High - [48, 53, ...], Jam [65, 72, ...]
- (b) Compute summary statistics for each group:  $Q_1$  (25% percentile),  $Q_2$  (50% percentile),  $Q_3$  (75% percentile), Interquartile Range  $IQR = Q_3 - Q_1$ , Lower whisker  $Q_1 - 1.5 \times IQR$  Higher whisker  $Q_3 + 1.5 \times IQR$ . Outliers are any value beyond the whiskers.
- (c) Boxplot
  - X-axis: Traffic Density (categorical)
  - Y-axis: Delivery Time (numeric)

- *Box*: from  $Q_1$  to  $Q_3$ , shows the middle 50% of delivery times.
- *Median line*: inside the box at  $Q_2$ .
- *Whiskers*: lines extending from the box to the minimum and maximum values within  $1.5 \times IQR$ .
- *Outliers*: individual points plotted beyond whiskers.

**Learning Outcome** Through this exercise students could build skills in hypothesis testing - a one sample Z-test, data visualization, and practical decision-making based on statistical insights.

© 2025 Rekha R. CE, FoT, UoD



## 10. ANOVA on Delivery Times

An e-commerce company wants to analyze how traffic conditions affect delivery times. You can use the the Kaggle dataset “[gauravmalik26/food-delivery-dataset/train.csv](https://www.kaggle.com/gauravmalik26/food-delivery-dataset/train.csv)” dataset containing delivery durations and the traffic density observed during each delivery. Handle missing values if any.

### Tasks

1. Compute summary statistics (mean, median, standard deviation) of delivery times for each traffic density group.
2. Visualize delivery times using a boxplot grouped by Traffic\_Density.
3. Group delivery times by traffic density. Write a function to test if the mean delivery times are significantly different across traffic density groups. Do not use any library to compute the  $F$ -statistic. You may use library function to get the  $p$ -value from the  $F$  table.
4. Note the interpretation of the results at a significance level of 5% in a Markdown cell: do traffic conditions significantly affect delivery performance?
5. Conduct Tukey’s HSD to identify which traffic levels differ significantly.
  - (a) *If ANOVA test fails*: perform a Tukey HSD test to identify which specific pairs of traffic density groups are significantly different.
  - (b) *If ANOVA test passes*:
    - Slightly adjust the mean of any group in the dataset so that differences between groups become more detectable.
    - Now perform the Tukey HSD test on the adjusted dataset to demonstrate which group pairs would differ if ANOVA were significant.

You may use library functions to do perform the Tukey test.

**Learning Outcome** By completing this exercise, students will be able to perform a one-way ANOVA to test for differences in mean delivery times across traffic density groups, interpret the  $F$ -statistic and  $p$ -value, and, if significant, apply Tukey HSD tests to identify which specific group pairs differ. They will learn to use Python libraries for these analyses and understand the conditional nature of Tukey testing.